

PROGRAMACIÓN WEB 3 · UNLaM

SignalR

Comunicación en tiempo real en el ecosistema ASP.NET Core

Trabajo Práctico de Investigación

Cristian Chazarreta · Elías Tucci · Franco Vargas · Jesica Salva · Martín Boga · Mauricio Galvan

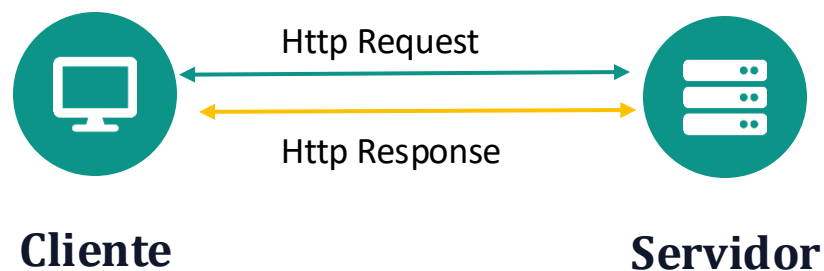
Introducción a SignalR (1/2)

SignalR es una biblioteca de **ASP.NET Core** desarrollada por **Microsoft** que permite la comunicación en tiempo real entre el servidor y los clientes. Gracias a SignalR, el servidor puede enviar actualizaciones de forma inmediata, sin que el usuario tenga que recargar la página.

Modelo Tradicional HTTP	SignalR
El cliente inicia la comunicación enviando una solicitud al servidor.	El cliente establece una conexión y el servidor puede enviar información cuando ocurre un cambio.
Cada actualización requiere una nueva solicitud HTTP.	La información se actualiza automáticamente en tiempo real.
El usuario puede tener que actualizar la página para ver cambios.	No es necesario actualizar la página.
La comunicación es puntual: solicitud y respuesta.	La comunicación es continua mientras la conexión permanezca activa.
Es adecuado para aplicaciones donde la información cambia poco.	Es ideal para aplicaciones que requieren información en tiempo real, como chats y notificaciones.

Introducción a SignalR (2/2)

Comunicación Http

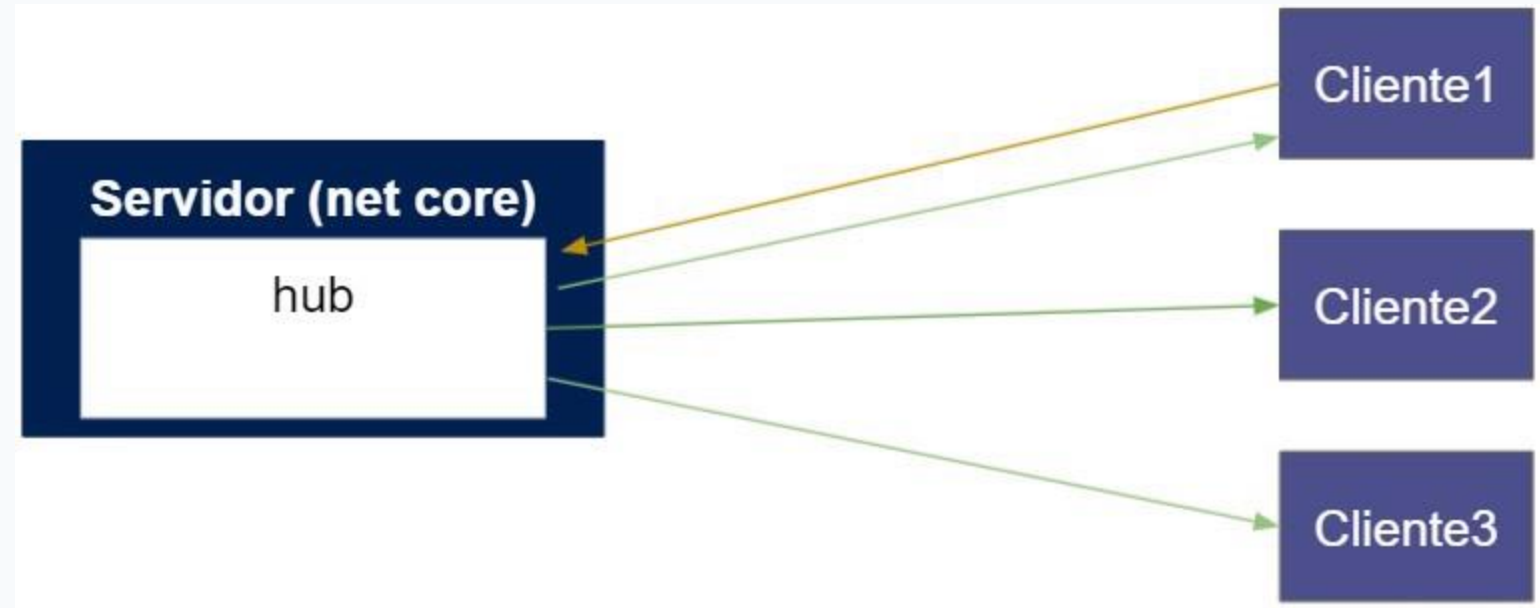


SignalR



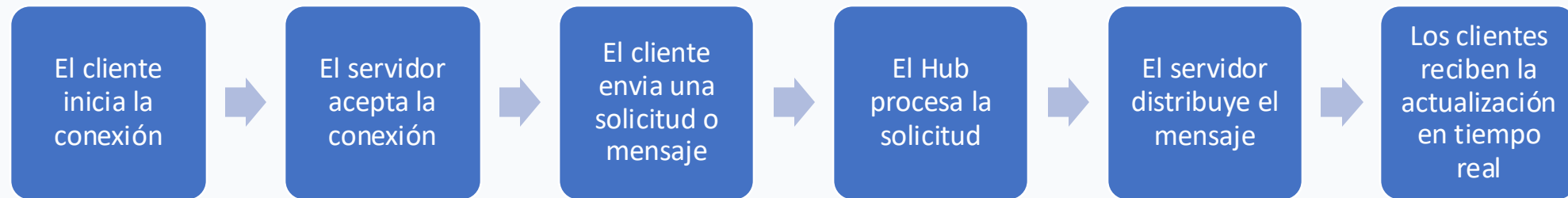
Arquitectura y funcionamiento (1/2)

- **Cliente:** Aplicación que se conecta al servidor que envía y recibe información en tiempo real e invoca métodos del Hub y recibe mensajes automáticamente.
- **Servidor:** Aloja SignalR, administra las conexiones y procesa las solicitudes de los clientes. Envía mensajes a clientes, usuarios o grupos.
- **Hub:** Componente central de SignalR. Este recibe solicitudes de los clientes y distribuye mensajes a uno o varios clientes



Arquitectura y funcionamiento (2/2)

Funcionamiento



Componentes Principales (1/3)



Hub

Componente central: administra la comunicación entre servidor y clientes. A través de él, los clientes invocan métodos del servidor.

```
using Microsoft.AspNetCore.SignalR;

public class ChatHub : Hub
{
    // Código
}
```



HubConnection

Representa la conexión que establece el cliente con el Hub del servidor, permitiendo enviar y recibir actualizaciones en tiempo real.

```
const connection = new signalR
    .HubConnectionBuilder()
    .withUrl("https://localhost:5001
              /chatHub")
    .build();

await connection.start();
```

Componentes Principales (2/3)



IHubContext

Permite enviar mensajes a los clientes desde cualquier parte de la aplicación, sin necesidad de estar dentro del Hub (por ejemplo, desde un Controller).

```
public class ChatController : Controller
{
    private readonly IHubContext<ChatHub> _hub;

    public async Task<IActionResult> Notificar()
    {
        await _hub.Clients.All.SendAsync(
            "RecibirMensaje", "Sistema", "Aviso");
        return Ok();
    }
}
```



Métodos del servidor

Definidos dentro del Hub, pueden ser invocados por los clientes. Contienen la lógica para procesar una solicitud o distribuir un mensaje.

```
public class ChatHub : Hub
{
    public async Task EnviarMensaje(
        string usuario, string mensaje)
    {
        await Clients.All.SendAsync(
            "RecibirMensaje",
            usuario, mensaje);
    }
}
```

Componentes Principales (3/3)



Métodos del cliente

Los métodos del cliente son aquellos que el servidor invoca para enviar información a las aplicaciones conectadas. En el cliente, estos métodos se registran mediante `connection.on()`, indicando el nombre del método que será ejecutado cuando el servidor envíe un mensaje

```
const conexion = new
signalR.HubConnectionBuilder()
  .withUrl("/hubs/chat")
  .build();

conexion.on("NuevaNotificacion",
function (data) {
  agregarNotificacion(data);
});
```


Gestión de conexiones

Cada cliente conectado recibe un `ConnectionId` único. SignalR expone eventos del ciclo de vida de la conexión para reaccionar cuando un cliente entra o sale.



ConnectionId

Identificador único asignado a cada conexión, usado por el servidor para identificar al cliente.

```
public override async Task
    OnConnectedAsync()
{
    var id =
        Context.ConnectionId;
}
```



OnConnectedAsync()

- Registrar usuarios
- Agregarlos a grupos
- Mostrar conexión

```
public override async Task
    OnConnectedAsync()
{
    Console.WriteLine(
        "Conectado");
    await base
        .OnConnectedAsync();
}
```



OnDisconnectedAsync()

- Eliminar usuarios
- Sacarlos de grupos
- Liberar recursos

```
public override async Task
    OnDisconnectedAsync(
        Exception? ex)
{
    await base
        .OnDisconnectedAsync(ex);
}
```

Protocolos de transporte

SignalR selecciona automáticamente el mecanismo más adecuado según las capacidades del cliente, el servidor y el navegador, en este orden de preferencia:

WebSockets

PREFERIDO

- Es el protocolo de transporte preferido por SignalR.
- Permite establecer una conexión bidireccional y permanente entre el cliente y el servidor.
- Una vez creada la conexión, ambas partes pueden enviar y recibir mensajes en cualquier momento sin necesidad de abrir nuevas conexiones.
- Ofrece la menor latencia y el mejor rendimiento.



Server-Sent Events

ALTERNATIVA

- Mecanismo que permite al servidor enviar información al cliente mediante una conexión HTTP que permanece abierta.
- Su comunicación es unidireccional.
- SignalR utiliza este protocolo únicamente cuando WebSockets no está disponible.



Long Polling

RESPALDO

- Es el mecanismo de respaldo utilizado por SignalR cuando no es posible emplear WebSockets ni Server-Sent Events.
- El cliente repite solicitudes HTTP que el servidor mantiene abiertas hasta tener datos. Simula tiempo real, pero genera más tráfico y rinde menos.

Implementación de SignalR (1/5)

1

Agregar la dependencia

Se incorpora mediante el paquete oficial de Microsoft.

```
dotnet add package  
    Microsoft.AspNetCore.SignalR
```

2

Registrar el servicio

Se registra SignalR en Program.cs.

```
builder.Services.AddSignalR();
```

3

Crear un Hub

Esta clase será el punto de comunicación entre el servidor y los clientes.

```
using Microsoft.AspNetCore.Authorization;  
using Microsoft.AspNetCore.SignalR;  
using Microsoft.EntityFrameworkCore;  
  
namespace WebApplication1.Hubs;  
  
[Authorize]  
public class OfertaHub : Hub  
{  
    private readonly MusicTradeDbContext _db;  
  
    public OfertaHub(MusicTradeDbContext db)  
    {  
        _db = db;  
    }  
}
```

Implementación de SignalR (2/5)

4

Configurar la ruta del hub

Luego se debe indicar a ASP.NET Core la dirección mediante la cual los clientes podrán conectarse al Hub en el Program.cs.

```
app.MapHub<OfertaHub>("/hubs/oferta");  
app.MapHub<ChatHub>("/hubs/chat");
```

5

Crear la conexión con el hub

Una vez configurado el Hub en el servidor, el cliente debe crear una conexión para poder intercambiar información en tiempo real.

```
var conexion = new signalR.HubConnectionBuilder()  
    .withUrl("/hubs/oferta")  
    .withAutomaticReconnect()  
    .build();
```

6

Iniciar la conexión

Una vez creada la conexión, el cliente debe iniciarla para comenzar a comunicarse con el Hub. El método `start()` establece la conexión entre el cliente y el servidor.

```
conexion.start().then(function () {  
    return conexion.invoke("JoinGroup", publicacionId);  
}).catch(function (err) {  
    console.error("Error ofertas SignalR:", err.toString());  
});
```

Implementación de SignalR (3/5)

7

Implementar los métodos del Hub.

En este proyecto, el método `EnviarOferta` recibe la oferta enviada por un usuario, la almacena mediante Entity Framework Core y posteriormente la distribuye a los clientes correspondientes.

```
public class OfertaHub : Hub
{
    public async Task EnviarOferta(int publicacionId, string mensaje)
    {
        Estado = EstadoOferta.Pendiente;

        _db.Ofertas.Add(oferta);
        await _db.SaveChangesAsync();

        await Clients.Group($"{publicacionId}").SendAsync("RecibirOferta", new
        {
            oferta.Id,
            oferta.UsuarioId,
            UsuarioNombre = $"{usuario.Nombre} {usuario.Apellido}",
            oferta.Mensaje,
            oferta.FechaCreacion,
            oferta.PublicacionId
        });
    }
}
```

8

Implementación de los métodos del cliente

En la aplicación desarrollada se utilizan para mostrar nuevas ofertas y notificaciones sin necesidad de actualizar la página.

```
conexion.on("RecibirOferta", function (oferta) {
    agregarOferta(oferta);
});
```

Implementación de SignalR (4/5)

9

Envío de mensaje

En la aplicación desarrollada, el envío de una oferta sigue un flujo compuesto por tres etapas: el cliente invoca un método del servidor, el servidor procesa la información y, finalmente, envía el resultado a los clientes correspondientes.

- **El cliente invoca al metodo del servidor**

Cuando el usuario escribe una oferta y presiona el botón **Enviar**, el cliente invoca el método `EnviarOferta()` definido en el Hub.

- **El servidor procesa la información y envia el mensaje**

Una vez recibida la solicitud, el método `EnviarOferta()` del Hub hace lo que tiene que hacer y luego la distribuye a todos los clientes que pertenecen al grupo de la publicación

- **El cliente recibe el mensaje**

Los clientes registran previamente el método que será ejecutado cuando el servidor envíe una nueva oferta

10

Implementacion de los métodos del cliente

En la aplicación desarrollada se utilizan para mostrar nuevas ofertas y notificaciones sin necesidad de actualizar la página.

```
conexion.on("RecibirOferta", function (oferta) {  
    agregarOferta(oferta);  
});
```

Implementación de SignalR (5/5)

10

Gestion de grupos

SignalR administra cada conexión mediante un identificador único denominado **ConnectionId**. Además, permite organizar las conexiones en **grupos**, facilitando el envío de mensajes únicamente a los usuarios que corresponden. En la aplicación desarrollada se utilizaron grupos para organizar las conversaciones, las publicaciones y las notificaciones personales.

```
public async Task JoinGroup(int publicacionId)
{
    await Groups.AddToGroupAsync(Context.ConnectionId, $"publicacion-{publicacionId}");
}
```

11

Gestion de conexiones

Cuando un usuario establece una conexión con el Hub, SignalR ejecuta automáticamente el método `OnConnectedAsync()`. Nuestra aplicación también implementa la **reconexión automática** utilizando `withAutomaticReconnect()`. Cuando la conexión se restablece, el cliente vuelve a incorporarse al grupo correspondiente.

```
public override async Task OnConnectedAsync()
{
    if (Context.User?.Identity?.IsAuthenticated == true)
    {
        var usuarioId = int.Parse(Context.User.FindFirstValue(ClaimTypes.NameIdentifier)!);
        await Groups.AddToGroupAsync(Context.ConnectionId, $"user-{usuarioId}");
    }
    await base.OnConnectedAsync();
}
```

Tipos de comunicación (1/2)

El servidor puede enviar información a distintos destinatarios según la necesidad: todos los clientes, una conexión puntual o un usuario autenticado.



Clients.All

Chat grupal, notificaciones generales, avisos del sistema.

```
await Clients.All.SendAsync(  
    "RecibirMensaje",  
    usuario, mensaje);
```



Clients.Client

Confirmar una operación o avisar solo al cliente que actuó (vía ConnectionId).

```
await Clients  
    .Client(connectionId)  
    .SendAsync(  
        "RecibirMensaje", mensaje);
```



Clients.User

Chat privado, notificaciones personales o mensajes directos a un usuario.

```
await Clients  
    .User(userId)  
    .SendAsync(  
        "RecibirMensaje", mensaje);
```


Tipos de comunicación (2/2)

Además, SignalR permite agrupar usuarios bajo un mismo nombre o excluir conexiones puntuales al momento de distribuir un mensaje.



Clients.Group

Envía mensajes a un conjunto de usuarios agrupados bajo un mismo nombre: chats grupales, equipos de trabajo, salas de juego o canales.

```
await Clients
    .Group("Musicos")
    .SendAsync(
        "RecibirMensaje",
        mensaje);
```



Clients.AllExcept

Envía un mensaje a todos los clientes conectados, excepto a uno o varios indicados: actualizar a todos menos a quien realizó la acción.

```
await Clients
    .AllExcept(connectionId)
    .SendAsync(
        "RecibirMensaje",
        mensaje);
```

Casos de uso



Chats



Marketplaces



Redes sociales



Juegos



Monitoreo



Notificaciones



Dashboards



Sist. colaborativos



Aplicación práctica desarrollada

Marketplace de instrumentos musicales con chat privado en tiempo real entre comprador y vendedor tras aceptar una oferta. Además, SignalR puede integrarse con IA para chatbots y asistentes virtuales en tiempo real.

Presentación de la práctica desarrollada

https://drive.google.com/file/d/1tPfeXsGsMXXaoX9dkUa8WzIx4Lv_JaJC/view?usp=sharing

Gracias Totales!

